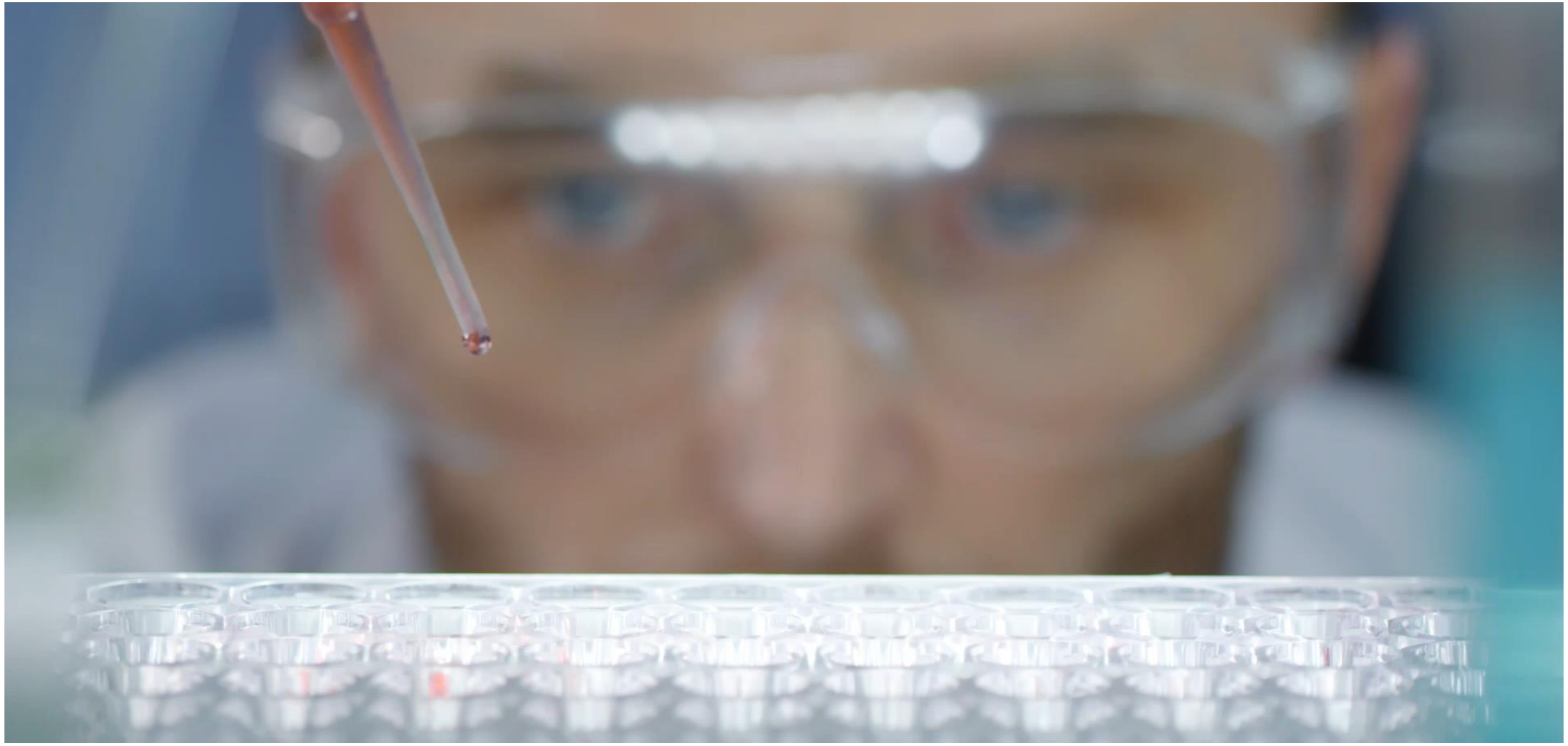
ENTORNOS DE DESARROLLO

UD3: DISEÑO Y REALIZACIÓN DE PRUEBAS



indice

1. Procedimientos de pruebas y casos de pruebas
2. Tipos de pruebas: pruebas funcionales
3. Pruebas no funcionales
4. Pruebas de caja blanca
5. Pruebas de caja negra
6. Herramientas de depuración de código
7. Planificación de pruebas
8. Calidad del software
9. Pruebas de rendimiento

Objetivos del testing de software

- Detectar y corregir errores.
- Proporcionar calidad y confiabilidad del software.
- Asegurar la correcta funcionalidad del producto.
- Evitar futuros errores.
- Facilitar la toma de decisiones para pasar a producción los desarrollos que no contengan errores.
- Cumplir con los requisitos del negocio y satisfacción del usuario.
- Evitar la aparición de nuevos defectos en el futuro que puedan afectar al software.

Tester

- Los probadores de software (también conocidos como testers, su denominación en inglés)

1. planifican y llevan a cabo pruebas de software de los ordenadores para comprobar si funcionan correctamente.
2. Identifican el riesgo de sufrir errores de un software, detectan errores y los comunican.

•**Un probador de software** necesita:Importantes habilidades de TIC.
•La capacidad de trabajar con calma bajo presión.
•Capacidad de organización.
•Pensamiento lógico.
•Capacidad de planificar el trabajo futuro.
•Prestar atención a los detalles.
•Ser capaz de redactar informes claros.
•Habilidades de comunicación oral.
•Habilidades de trabajo en equipo

Casos de pruebas

- Hay que evaluar primero los **casos de pruebas**. Ejemplo una aplicación sencilla de entrada de datos o una web donde hay que registrarse.
- El primer software que se suele sacar es una versión beta, que exige un proceso de **beta testing.**
- Otro aspecto es **preparar las maquinas** sobre las que hay que hacer las pruebas.
- El siguiente paso es la **ejecución de las pruebas.**
- **Nos vamos a encontrar dos tipos de pruebas:**
  - **Funcionales** que identifican la funcionalidad del sistema
  - **No funcionales** que tienen un carácter más técnico.

Tipos de pruebas: pruebas funcionales

- **Pruebas unitarias (Unit testing):** Es una forma de probar una unidad o fragmento de código más pequeño que se puede aislar lógicamente en un sistema.
- **Pruebas de integración (Integration testing):** Prueban la integración o interfaces entre componentes, interacciones con diferentes partes del sistema, como sistema de archivos y hardware, sistema operativo,o interfaces entre sistemas.
- **Pruebas de sistema (System testing):** Se evalúa cómo los diferentes componentes de una aplicación interactúan juntos en el sistema o aplicación completa e integrada.
- **Pruebas de sanidad (Sanity testing):** Se trata de una prueba rápida y básica para determinar si se comporta correctamente una aplicación o componente en particular.
- **Pruebas de humo (Smoke testing):** Es el proceso de prueba inicial que verifica la funcionalidad principal de un programa para garantizar que esté listo para más pruebas.
- **Pruebas de interfaz (Interface testing**): Verifica la comunicación entre dos sistemas de software diferentes comprobando la autenticación de la conexión establecida.
- **Pruebas de regresión (Regression testing):** Garantiza que una aplicación siga funcionando según lo esperado después de producirse alguna actualización, cambio o mejora de código.
- **Pruebas de aceptación (Beta/acceptance testing):** se realiza en un producto o prototipo antes de su comercialización o entrega, para decidir si se han cumplido las especificaciones o el contrato. Se asegura de que la calidad y el diseño del producto cumplan con los requisitos en términos de funcionalidad, usabilidad, durabilidad y seguridad.

Pruebas no funcionales

- **Pruebas de rendimiento (Permormance testing):** Evalúa el rendimiento, la velocidad o capacidad de respuesta de la aplicación que se prueba bajo la carga de trabajo requerida.
- **Prueba de carga (Load testing):** Prueba el comportamiento de una aplicación bajo una gran carga de trabajo.
- **Pruebas de estrés (Stress Testing):** Las pruebas de estrés determinan la solidez del software al evaluar el funcionamiento normal en condiciones de carga extremadamente pesada, garantizando así que el software no se bloquea en situaciones críticas.
- **Pruebas de volumen (Volume Testing):** Prueba el rendimiento del software cargando cuando procesa grandes volúmenes de datos.
- **Pruebas de seguridad (Security Testing)**: Verifican si el sistema está protegido contra ataques repentinos o deliberados de fuentes internas y externas.
- **Pruebas de compatibilidad (Compatibility Testing):** Comprueban si la aplicación es compatible con diferentes entornos.
- **Pruebas de instalación (Install Testing):** Verifica si un software funciona según las expectativas después de la instalación.
- **Pruebas de recuperación (Recovery Testing):** Determina si un sistema puede recuperarse de fallas o no.
- **Pruebas de confiabilidad (Reliability Testing):** Garantiza que el software funciona de manera consistente realizando una tarea sin fallar dentro un período específico.
- **Pruebas de usabilidad (Usability Testing):** Testean la facilidad de uso del usuario en términos de operación, aprendizaje y preparación de entradas y salidas.
- **Pruebas de conformidad (Compliance Testing):** Determina si un programa o sistema de software cumple con un conjunto definido de estándares internos o externos antes de su lanzamiento a producción.
- **Pruebas de localización (Localization Testing):** Verifican el comportamiento de un producto de acuerdo con los entornos locales o culturales específicos.

Pruebas de caja blanca

- Se realiza un análisis del código.

PRUEBA DE CUBRIMIENTO → Ejecutar al menos una vez el código. Cubriendo todos los caminos posibles, (número ciclomático) por tanto con tantas pruebas como sea necesario.

PRUEBA DE CONDICIONES → Se usarán varios casos de pruebas para determinar las posibles solcuiones de cada condición.

PRUEBA DE BUCLES → Se realizarán casos de pruebas repetiendo por ejemplo un número x veces.

Pruebas de caja negra

- Se realiza un análisis de la interfaz.

PRUEBA DE CLASES DE EQUIVALENCIA DE DATOS

Por ejemplo para comprobar el nombre y usuario en un formulario.(mayúsculas, minúsculas, etc) de manera que comprobamos a través de clases de equivalencia.
Ejemplo: queremos que el usuario tenga mayúsculas, minúsculas, carácter y un número
Clase valida: Gandalf_1234, clase invalida: gandalf1234

PRUEBA DE VALORES LÍMITE

Se utilizan pruebas para validar si el programa funciona correctamente, por ejemplo la nota tiene que ser menor que 11, hay que probar que al poner 11, nos lo admita.

PRUEBA DE INTERFACES

Como testear una interfaz

1º. Seguir el manual de usuario.
2º Realizar casos de prueba.

Testear la usabilidad

Se trabaja desde el punto de vista del usuario.

Testear la usabilidad

- Accesibilidad usuarios con discapacidad.
- Accesibles por frameworks de test automatizados. Ejemplo Selenium

Herramientas de depuración de código: JUNIT

- **JUnit** es un Framework, que permite la realización de test repetibles.(pruebas de regresión)

Ejemplo libro de Sintesis:

crear una clase Junit en Java.
Deshabilitar un test
Trabajar con excepciones
Crear un conjunto de test

Ampliación: JUnit en Visual Studio Code https://code.visualstudio.com/docs/java/java-testing

https://www.eclemma.org/installation.html instalar esta aplicación para EclEmma **es una herramienta para Eclipse que permite visualizar la cobertura del código de los tests unitarios que hemos realizado**. Esta herramienta se utiliza cuando estamos realizando pruebas de Caja Blanca, es decir, tenemos el código fuente que queremos probar disponible.

Inicializadores y finalizadores en JUnit4

```
@BeforeClass
public static void setUpClass() {
}

@AfterClass
public static void tearDownClass() {
}

@Before
public void setUp() {
}

@After
public void tearDown() {
}
```

Inicializadores y finalizadores en JUnit4

Inicializador de la clase. **@BeforeClass**. Se ejecutará solamente una vez y antes de cualquiera de los otros métodos de la clase de test.

Finalizador de la clase. **@AfterClass**. Solamente se ejecutará una vez, justo después de la ejecución de los métodos de la clase test.

Inicializador del test. **@Before**. Marca el método que se va a ejecutar antes de cada test.

Finalizador del test. **@After**. Marcaría el método que se va a ejecutar después de cada test.

Deshabilitar un test en JUnit

Colocar **@ignore** en una línea antes del **@test**

```
@Ignore
@Test(timeout=1000)
 public void testFactorialTimeout() {
   System.out.println("factorial con timeout");
    int numero = 300;
```

Junit. Trabajando con excepciones

```
public int factorial(int numero) {
    int factorial = numero;
    if (numero < 0){
        throw new IllegalArgumentException("Factorial negativo!");
    }
    for(int i =(numero - 1); i > 1; i–){
        factorial = factorial * i;
    }
    return factorial;
}
```

Junit. Trabajando con excepciones

```
if (numero < 0){
    throw new IllegalArgumentException("Factorial negativo!");
}
```

Si el número es menor que cero -> saltará una excepción.

Para que el test no presente ningún fallo:

```
@Test(expected=IllegalArgumentException.class)
public void testFactorial() {
.......
```

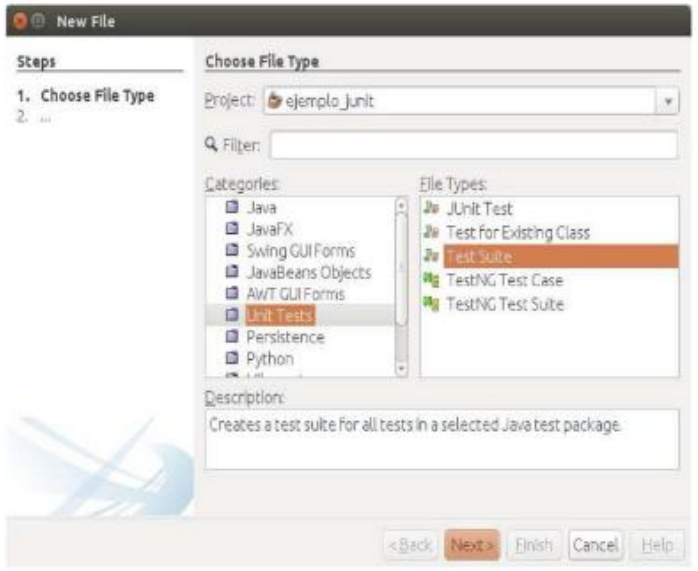
JUNIT- SUITE DE TEST

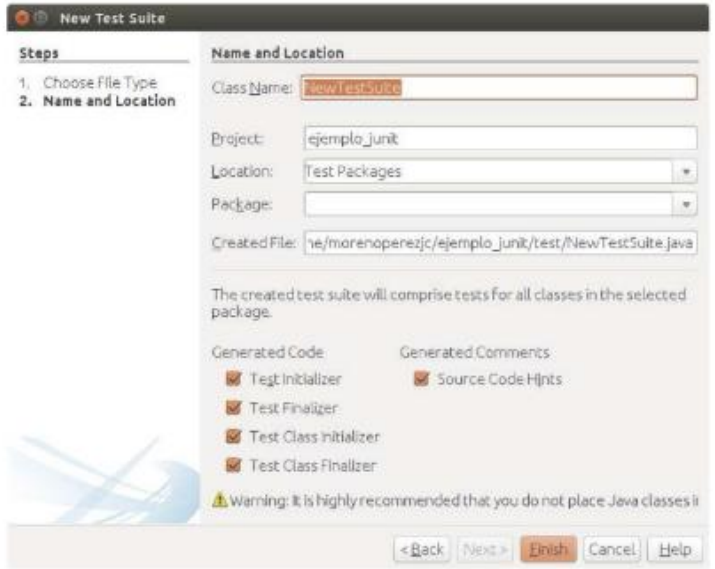
Junit. Suite de test o conjuntos de test

Se agrupa una serie de test en un conjunto.

Sirve para lanzar a la vez una serie de test.

Junit. Suite de test





JUNIT- SUITE DE TEST

Junit. Suite de test

En @Suite.SuiteClasses se incorporarán todos los test de pruebas que se desee y en el orden que se necesiten.

Basta con separarlos con comas como por ejemplo:

value={clase1Test.class, clase2Test.class}

Planificación de pruebas

1º PRUEBAS UNITARIAS

Se realizan sobre todo en las primeras fases de diseño y desarrollo.
En POO a nivel de objetos, luego a nivel de paquete o librería.

2º PRUEBAS DE INTEGRACIÓN

Al final de la fase de diseño, una vez realizado las pruebas individuales o unitarias, puede haber pruebas ascendentes o descendentes, es decir de los modulos específicos a los generales o viceversa. En el primer caso hay que crear modulos, objetos, clases de prueba y en el caso descendente modulos de prueba o programas de pruebar para probar los modulos individuales.

3º PRUEBAS DE VALIDACIÓN

Prueban el sistema completo, y que se cumplan los requisitos, también se prueba la carga y se realizan pruebas de estrés para ver como funciona ante cualquier eventualidad. Se dividen en dos tipos, ALFA-ENTORNO CONTROLADO Y ESPECIFICADO BETA ENTORNO NO CONTROLADO

4º AUTOMATIZACION DE PRUEBAS

Automatizar las pruebas, permite tener un historial tras repetir las pruebas.

AUTOMATIZACIÓN DE PRUEBAS

Automatización de pruebas

Repetir las mismas pruebas tras realizar mantenimientos, modificaciones o correcciones del software.

Conservar los datos, set de pruebas, programas y módulos de prueba para un futuro.

Documentar y almacenar en un repositorio para su ejecución automática posterior.

Selenium/Katalon

Tenga en cuenta lo siguiente:

Selenium IDE -> Para Firefox anteriores al 55

Katalon Recorder -> Para Firefox 55 y superiores y también para Chrome.

SELENIUM-KATALON

Selenium/Katalon

Realizar **juegos de pruebas** para aplicaciones web.

Grabar, editar e ir **depurando** los casos de prueba que se podrán ejecutar de forma automatizada.

Permite **grabar acciones** para luego editarlas o bien ir creándolas desde cero.

Pruebas funcionales de **regresión**.

Selenium

Ventajas de Selenium destacan:

- El **autocompletado** de los comandos.
- Se pueden utilizar **breakpoint** y herramientas de depuración.
- Se pueden **almacenar los conjuntos** de pruebas en muchos formatos.
- Se pueden **referenciar objetos del DOM** en base a su ID (o también por su nombre o a través de XPath).

Selenium. Ten en cuenta...

name: nombre del objeto.

id o identificador: Selenium-IDE utiliza esta opción por defecto al grabar las pruebas.

dom: el DOM es el document object model.

xPath: al igual que el DOM, el xPath depende de la estructura del documento y permite referenciar los elementos de una página web mediante una ruta (absoluta o relativa).

Selenium. Componentes

Selenium client es el cliente de Selenium. Será necesario para crear pruebas Junit con Selenium.

Selenium IDE es el plugin que se instala en Firefox para grabar pruebas.

Selenium Server es el servidor de Selenium desde donde se realizan las pruebas.

INSTALACIÓN DE SELENIUM

Selenium. Instalación

Página oficial de Firefox:
https://addons.mozilla.org/es/firefox/addon/selenium-ide/

Página oficial de Selenium: http://www.seleniumhq.org/download/

Selenium

- Ejercicio: instalar Selenium en Eclipse.

1. Abra Eclipse y vaya a "Help" -> "Eclipse Marketplace".
2. Busque "Selenium" en el cuadro de búsqueda y haga clic en el botón "Go".
3. Seleccione "Selenium IDE" y haga clic en "Install".
4. Siga las instrucciones en la pantalla para completar la instalación.
5. Reinicie Eclipse para que los cambios surtan efecto.
6. Para utilizar Selenium con Java, debe descargar e instalar las bibliotecas Selenium para Java. Puedes hacerlo desde https://www.seleniumhq.org/download/
7. Para importar las bibliotecas en Eclipse, vaya a "File" -> "Import" -> "Maven" -> "Existing Maven Projects" y seleccione la carpeta donde descargó las bibliotecas Selenium.

Selenium ejercicios

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class SeleniumExample {

    public static void main(String[] args) {
        // Establecer la ruta del controlador del navegador
        System.setProperty("webdriver.chrome.driver", "path/to/chromedriver");
        // Crear una nueva instancia del controlador del navegador
        WebDriver driver = new ChromeDriver();
        // Navegar a la página web deseada
        driver.get("https://www.example.com");
        // Obtener el título de la página
        String title = driver.getTitle();
        // Imprimir el título de la página
        System.out.println("Page title: " + title);
        // Cerrar la ventana del navegador
        driver.quit();
    }
}
```

- Para poder utilizar el ejercicio anterior, previamente hay que descargar el driver de Google Chrome
- Para descargar e instalar el controlador de Google Chrome en Eclipse, siga estos pasos:
- Descargue el controlador de Google Chrome desde el siguiente enlace: https://sites.google.com/a/chromium.org/chromedriver/downloads
- Una vez descargado, descomprima el archivo y copie el ejecutable del controlador a una carpeta en su sistema.
- En Eclipse, abra el proyecto en el que desea utilizar Selenium con Google Chrome.
- En el archivo Java que contiene el código de Selenium, establezca la ruta del controlador utilizando el siguiente código:
- java
- System.setProperty("webdriver.chrome.driver", "path/to/chromedriver");
- Reemplace "path/to/chromedriver" con la ruta completa al ejecutable del controlador que copió en el paso 2.

- Asegúrese de tener las bibliotecas Selenium para Java en su proyecto, si no es así importelas.

- Ejecute su proyecto y Selenium utilizará el controlador de Google Chrome para automatizar las tareas de navegación.

KATALON EJERCICIO

- Aquí tienes un ejemplo básico de cómo utilizar Katalon para automatizar la navegación en un sitio web:
- Descargue e instale Katalon Studio en su sistema siguiendo las instrucciones en su sitio web oficial.
- Inicie Katalon Studio y cree un nuevo proyecto.
- En la vista de "Test Cases", haga clic en el botón "New Test Case" para crear un nuevo caso de prueba.
- En la ventana emergente, ingrese un nombre para el caso de prueba y haga clic en "OK".
- En la ventana de edición de casos de prueba, haga clic en el botón "Record" para comenzar a grabar las acciones del navegador.
- En el navegador, vaya a la página web que desea automatizar.
- Realice las acciones deseadas en la página web (por ejemplo, haga clic en un enlace, ingrese datos en un formulario, etc.).
- Una vez que haya terminado, haga clic en el botón "Stop" en Katalon Studio para detener la grabación.
- En la vista de "Test Cases", haga clic en el botón "Run" para ejecutar el caso de prueba grabado.
- Observe los resultados en la vista "Test Suites" y revise los detalles de las acciones realizadas en el navegador.
- Este ejemplo es básico y solo muestra cómo automatizar la navegación en una página web utilizando Katalon Studio, pero hay muchas otras funciones y opciones disponibles en la herramienta para automatizar una variedad de tareas y validar resultados.

Calidad del software

Características:

- **Idoneidad funcional**. El grado o nivel en que el software resultante proporciona funciones que satisfacen las necesidades establecidas para determinadas condiciones.
- **Fiabilidad.** El grado en que un sistema o componente software realiza sus funciones bajo determinadas condiciones y período de tiempo.
- **Eficiencia en el desempeño**. El rendimiento en relación con la cantidad de recursos utilizados en las condiciones establecidas.
- **Usabilidad**. El grado o nivel en que el software permite ser entendido, aprendido, usado y fácil para el usuario.
- **Seguridad.** El grado de protección de la información y los datos para que las personas o los sistemas no autorizados no puedan leerlos o modificarlos y que permita el acceso y uso a las personas o sistemas autorizados.
- **Compatibilidad**. El grado en que dos o más sistemas o componentes pueden intercambiar información y/o realizar sus funciones.
- **Mantenibilidad.** El grado de eficacia y eficiencia con que el software puede ser. Modificado y/o actualizado.
- **Transferibilidad.** El grado o nivel en que un sistema o componente transferido de un hardware, software u otro entorno operativo a otro.

Pruebas de rendimiento

- Se comprueba como el software realiza una tarea determinada bajo unas condiciones concretas. Se determina aspectos como uso de recursos, escalabilidad, fiabilidad etc..

PRUEBAS DE CARGA → Se simulan una serie de peticiones, se evaluan tiempos de respuesta, y se evaluan con varios tipos de carga, baja, media, alta.

PRUEBAS DE ESTRES → Se realizan cargas extremas.

PRUEBAS DE ESTABILIDAD → Se realizan cargas continuadas

PRUEBAS DE PICOS → Se realizan cargas variables.

Benchmark

- Aplicación o conjunto de aplicaciones, para realizar pruebas de rendimiento de un sistema.

Existen cuatro categorías generales de pruebas de comparación:

- ✓ *Pruebas de aplicaciones base*: que se encargan de ejecutar y cronometrar los tiempos de estas.
- ✓ *Pruebas* playback: que usan llamadas al sistema durante actividades específicas de una aplicación como uso de disco o llamadas a rutinas de gráficos, ejecutándolas aisladamente.
- ✓ *Pruebas sintéticas:* que enlazan actividades de la aplicación en subsistemas específicos.
- ✓ *Pruebas de inspección:* que no intentan imitar la actividad, sino que las ejecuta directamente en su entorno productivo.

Aplicaciones Hardinfo, Winstone o Winbench de ZDNet

Medidas o métricas de calidad del software

- Los criterios de calidad se definen al principio del proyecto:

1. Número de errores por un número determinado de líneas de código.
2. Número medio de revisiones realizadas a una función o modulo del programa.

RTF

Es una revisión técnica formal, con la intención de descubrir errores, en la funcionalidad, lógica, implementación, diseño, etc.. Las realiza el responsable de calidad o SQA, se revisa una parte del software, se hace un informe y se tiene que llevar a cabo las modificaciones.

Ejercicios

1. Descargar e instalar Hardinfo en Linux
   1. https://ubunlog.com/hardinfo-verifica-informacion-de-hardware/
2. Ejercicios en un IDE, con varios aplicaciones de JAVA, usando Junit.
3. Descargar y usar aplicaciones para realizar pruebas.
   1. https://saasradar.net/herramientas-prueba-de-software/
   2. Usar selenium con java
      1. https://www.youtube.com/watch?v=iGxG9U56hvQ
      2. https://spa.myservername.com/webdriver-entire-setup
4. Trabajo sobre la automatización de pruebas.
5. Aplicaciones Hardinfo, Winstone, Winbench de ZDNet

- Aplicaciones para automatizar pruebas:
- https://ciberninjas.com/10-mejores-herramientas-pruebas-ui